MemDB2 C DLL Benchmark Results

mdb2bench, 11 November 2026 · x64 Release DLL · 1 million records · 4 readers + 4 writers

> **This report replaces MemDB2_Benchmark_Results_2026-11-10.pdf.** The previous report was generated with a Release build of the C DLL that still had extra debug checking compiled in. Its Release configuration didn't turn assertions off, so every `{$IFOPT C+}` consistency check ran, and every row, row version and pin was registered with a global debug object tracker. Its figures overstate MemDB2's costs and should no longer be used. This report uses a Release build with assertions off, and readers in READ_WRITE_SHARED mode now end with a rollback instead of a commit, since they change nothing.

0.22–0.25 ms
single write transaction (insert, delete), isolated

28–41 µs
single read transaction, isolated, either access mode

29.4k txn/s
mixed 4 + 4 workload, all transactions READ_WRITE_SHARED

-36%
throughput with async checkpoints running; worst case 125 ms → 1.35 s

Summary

All figures come from an old Ivy Bridge laptop, with every call going through the C DLL. Expect modern hardware, and native Delphi use, to be considerably faster (see Caveats).

- **Writes cost about 0.23 ms each, updates about twice that.** Across the three runs, insert, delete by ID and delete by expiry take 0.22–0.25 ms (460,000–527,000 CPU cycles) as single transactions. State transition and recycle, which update a row in place, take 0.40–0.41 ms (864,000–879,000 cycles).
- **Reads cost 28–41 µs in either access mode** (61,000–90,000 cycles). With readers ending in a rollback, READ_WRITE_SHARED reads now cost the same as READ_SHARED ones.
- **Everything is about 2.5 times faster than the previous report showed.** Writes and updates take 2.5–2.8 times fewer cycles, memory per row fell from about 2,130 to 1,625 bytes, and mixed-workload throughput rose 2.4–4.5 times.
- **The access modes now give similar totals with different splits.** With READ_SHARED readers the database ran 27,414 txn/s, with writers getting 3,665/s. With all transactions READ_WRITE_SHARED it ran 29,371/s, and writers nearly doubled to 7,022/s.
- **Write tails are still long.** p99.99 is 14–22 ms for every write in the isolated phase, against 0.5–1.1 ms for reads.
- **Asynchronous checkpoints cost a third of the throughput, mostly from writers.** Throughput fell 36% and inserts/s fell 55%. p99 rose 8× and the worst case to 1.35 s. The one checkpoint timed during the measurement took 14.2 s, against 2.5 s for a synchronous checkpoint of the same data.

Test setup

Machine	Laptop, Intel Core i7-3632QM (Ivy Bridge, 2012), 2.2 GHz, 4 cores / 8 threads; invariant TSC (2.195 GHz)
Database	MemDB2 C DLL, x64 Release, assertions off, without the benchmark allocation hooks. All calls go through the C API shim (MemDB2CApi), not native Delphi
Data	1,000,000 rows of about 128 bytes (Id, Expiry, State, Payload blob); unique index on Id, index on Expiry
Transactions	Writers READ_WRITE_SHARED, lazy write journal, repeatable read isolation. Readers READ_SHARED (read run) or READ_WRITE_SHARED (rws runs). Readers end every transaction with a rollback
Isolated phase	Each of 7 operations alone on one thread, 200,000 transactions each
Mixed phase	4 reader + 4 writer threads, unpaced, 30 s after a 5 s warm-up. Per insert: delete by ID p = 0.5, state transition p = 0.25, recycle p = 0.1; expiry sweep every 500 µs. Readers: find by ID, with find earliest every 10th read
Runs	`plain_tests.bat`: `--records 1M --reader-mode read` (read); … `--reader-mode rw` (rws); … `--reader-mode rw --ckpt async` (rws-ckpt, back-to-back asynchronous checkpoints during the mixed phase)

Loading the 1M rows took 125–144 s (6,900–8,000 rows/s, in transactions of 10,000 rows) and used about 1,625 bytes of process memory per row, roughly 13 times the logical row size. A synchronous checkpoint of the loaded data took 2.5–2.6 s. An earlier set of runs was discarded because another application on the machine was disturbing the results.

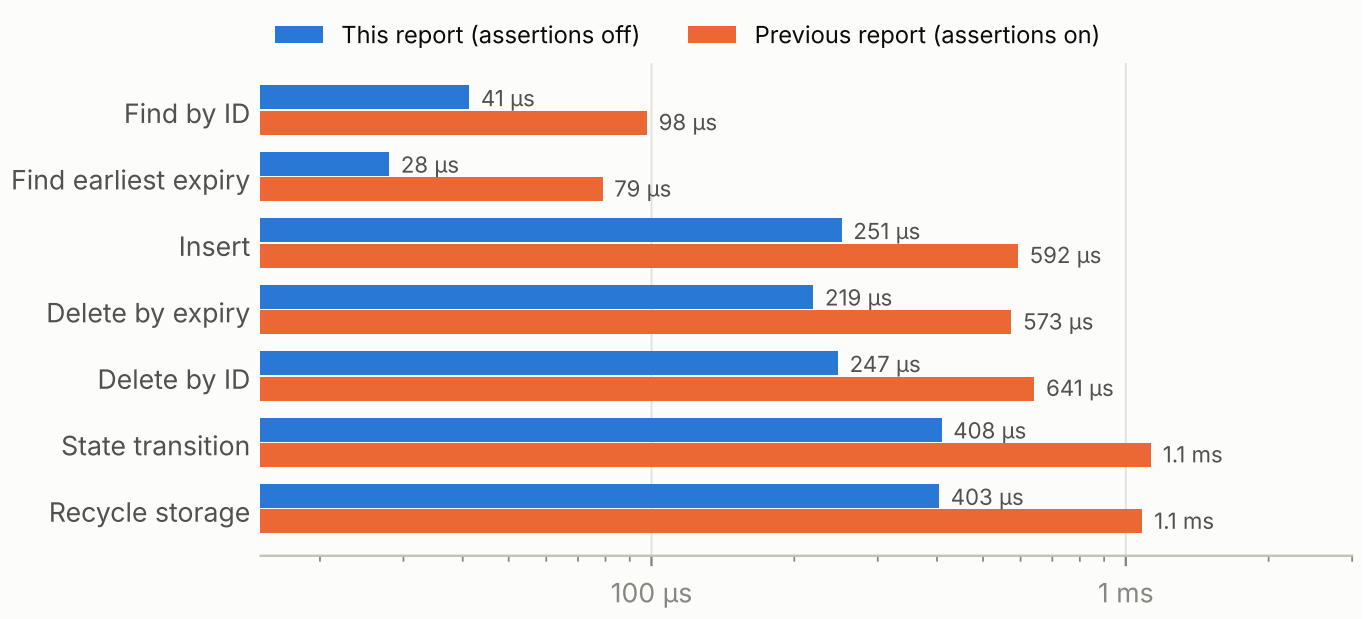
Changes since the previous report

The same runs, on the same machine, before and after turning assertions off. The previous report's READ_WRITE_SHARED readers also committed instead of rolling back, which accounts for part of their improvement.

Measure (rws run unless stated)	Previous report	This report	Change
Insert, isolated (thread cycles)	1,246,932	498,283	÷2.5
Find by ID, isolated (thread cycles)	206,848	89,865	÷2.3
Find earliest expiry, isolated (thread cycles)	158,697	60,899	÷2.6
Delete by ID, isolated (thread cycles)	1,394,113	526,909	÷2.6
Delete by expiry, isolated (thread cycles)	1,246,461	468,744	÷2.7
State transition, isolated (thread cycles)	2,458,736	878,833	÷2.8
Recycle storage, isolated (thread cycles)	2,362,202	868,968	÷2.7
Mixed throughput, read run (txn/s)	11,584	27,414	×2.4
Mixed throughput, rws run (txn/s)	6,512	29,371	×4.5
Mixed throughput, rws-ckpt run (txn/s)	6,224	18,699	×3.0
Memory per row (bytes)	2,128	1,623	-24%
Preload of 1M rows (s)	261	127	-51%

Figure 1. Mean latency per transaction, isolated, rws run

Logarithmic scale. This report against the previous report's build with assertions on.

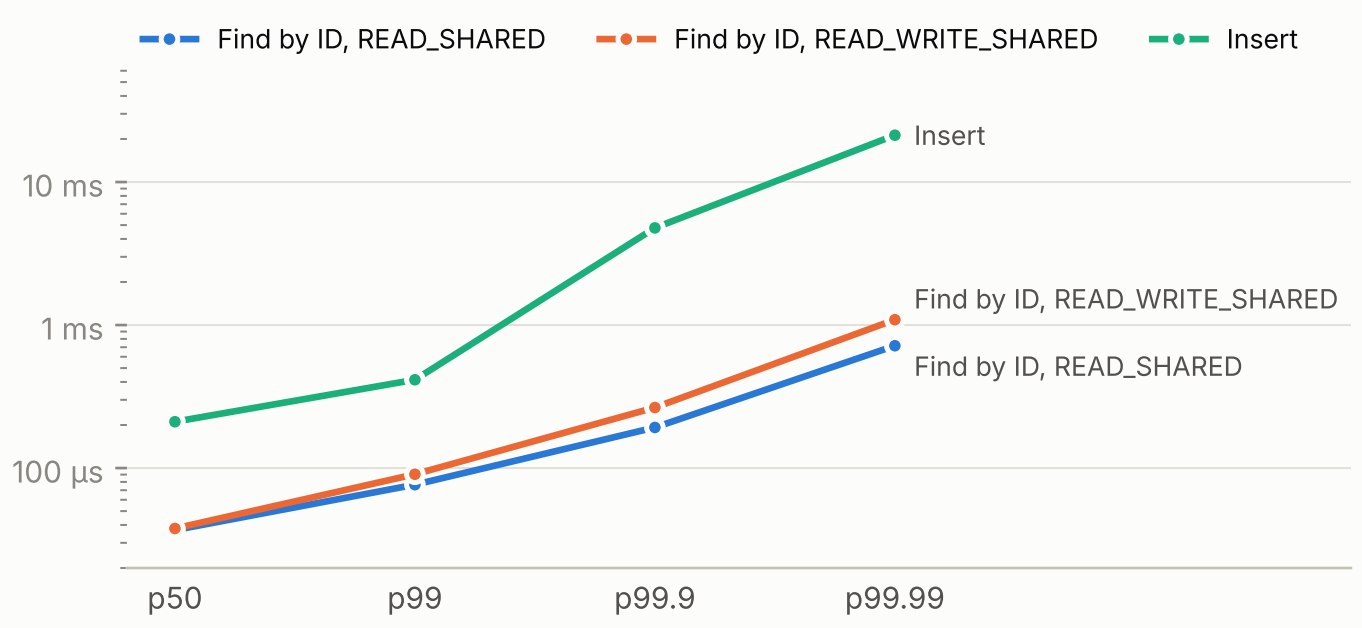


Isolated operations

Each operation runs as a complete transaction on one thread, with nothing else running, so these are the base costs of the engine and the C API. The three runs agree closely: their means are within 10% of each other for every operation (see Appendix).

Figure 2. Latency percentiles, isolated

Logarithmic scale. Find by ID in both access modes, and insert.

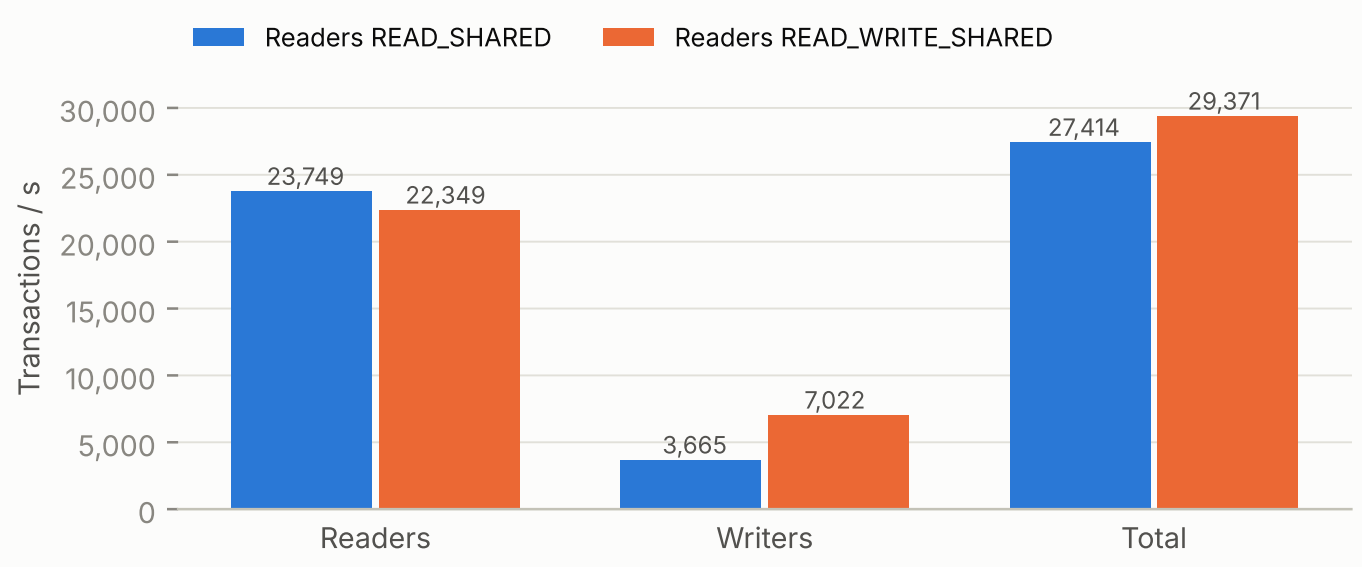


Reads stay below 1.1 ms at p99.99 in both modes. Writes reach 14–22 ms at p99.99, almost regardless of the operation's own cost, which points at something periodic of roughly constant length in the write path. With assertions off this is still the first thing to profile. Single-run maxima of up to about 1 s weren't consistent between runs, so they're left out of this comparison.

Mixed workload

Figure 3. Throughput by role, mixed workload

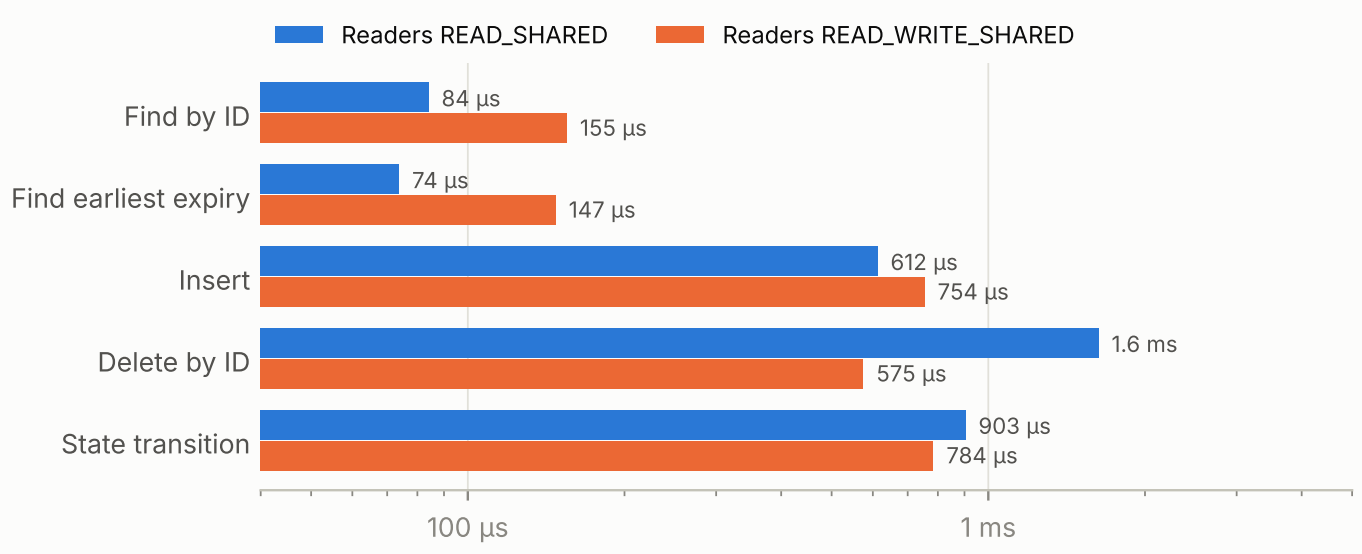
Transactions per second over the 30 s measurement.



READ_SHARED and READ_WRITE_SHARED transactions take the database's read/write lock in different modes, so they run in alternating batches. With readers in READ_SHARED, reads dominate: 23,749 reads/s against 3,665 writes/s. With every transaction in READ_WRITE_SHARED there is one mode; writers almost double, and the total rises slightly.

Figure 4. Median latency per operation, mixed workload

Logarithmic scale. Delete by expiry and recycle are left out: they found nothing to do (see Caveats).



The same effect shows in the medians. Reads are about twice as fast with READ_SHARED readers (84 µs against 155 µs for a find by ID), while writers wait longer: a delete by ID's median is 1.63 ms against 0.57 ms. Total CPU per transaction was 236,503 cycles (READ_SHARED) and 385,505 (READ_WRITE_SHARED), reflecting the different mix. No transaction hit a concurrency clash in any run.

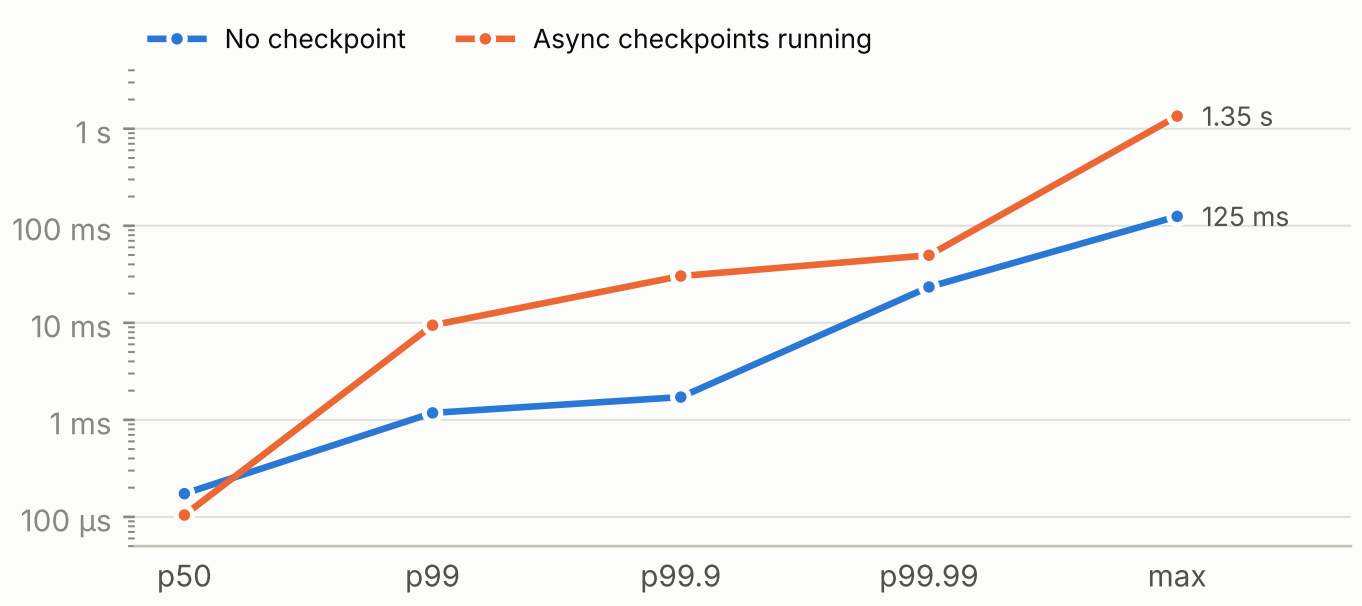
In the read run, every operation type had a worst case of about 155–160 ms, suggesting one stall that held up all threads at once. The rws run didn't show it.

Asynchronous checkpoints

The rws-ckpt run repeats the rws run with a separate thread writing asynchronous checkpoints back to back during the mixed phase. A checkpoint was running for 45% of the measured time. The comparison is between the two runs.

Figure 5. Latency percentiles, all mixed-phase transactions

Logarithmic scale. rws (no checkpoint) against rws-ckpt (asynchronous checkpoints running).



Measure	No checkpoint	Async checkpoints	Change
Throughput (txn/s)	29,371	18,699	-36%
Writer transactions/s	7,022	3,153	-55%
Inserts/s	2,863	1,287	-55%
p50	174 µs	104 µs	-40%
p99	1.18 ms	9.43 ms	×8.0
p99.9	1.72 ms	30 ms	×17.7
p99.99	23 ms	50 ms	×2.1
Max	125 ms	1.35 s	×11
Process CPU cycles per transaction	385,505	721,458	+87%

Writers keep committing during an asynchronous checkpoint, as designed, but at about half the rate: inserts/s fell 55%. The median transaction got faster (174 µs → 104 µs), only because the mix shifted towards cheap reads. From p99 upwards latency rises sharply: p99.9 reached 30 ms and the worst insert took 1.35 s.

The one checkpoint timed during the measurement took 14.2 s, about 5.6 times a synchronous checkpoint of the same data (2.5 s). The checkpoint thread used 23.1 billion cycles while measuring, about 11 s of CPU, so the checkpoint is largely CPU-bound rather than waiting on the disk. With only one timed checkpoint these figures are rough; the previous report's checkpoints, with assertions on, took about 13 s.

Caveats

- **Old hardware.** The tests ran on a 2012 Ivy Bridge laptop (Core i7-3632QM, 2.2 GHz). Absolute figures are unlikely to be representative of modern hardware; the comparisons between configurations are more meaningful than the raw numbers.
- **Every call goes through the C DLL shim.** Each transaction makes about 6–10 C API calls. Each call validates and looks up its handle in a handle table guarded by a single lock, and fields are named by string on every read and write. The shim is likely to be considerably slower than using MemDB2 natively from Delphi, and the shared lock may also add contention in the mixed workload.
- **Expiry processing wasn't exercised in the mixed phase.** Records expire 1,386,294 inserts after their own insert, but only 41,000–90,000 inserts happened during each mixed phase, so nothing expired. Every delete-by-expiry and recycle transaction in that phase was a miss: an empty probe of the expiry index, then a rollback. A shorter `--lifetime` is needed to measure expiry under load. In the isolated phase, expired rows were available and those transactions did real work.
- **Misses are included in the latencies.** About 43% of mixed-phase finds by ID, and 42% of deletes by ID, targeted rows already deleted.
- **Checkpoint overlap.** In the rws-ckpt run the benchmark counted every measured transaction as overlapping a checkpoint, although a checkpoint was running only 45% of the time. Treat its figures as an average over the whole phase rather than as checkpoint-only behaviour.
- **One run per configuration.** The isolated phase is identical in all three runs, and the agreement between them (within 10%) gives a rough idea of run-to-run variation.
- **Not covered here:** the baseline engine (plain data structures, to measure MemDB2's overhead), flush-per-commit journalling, x86, the specified 5 million rows, a database larger than RAM, and allocations per transaction (the DLL was built without the counting hooks).

Appendix: data

Isolated phase, read run (readers READ_SHARED)

Operation	txn/s	mean	p50	p99	p99.9	p99.99	max	thread cycles	process cycles
Insert	4,300	233 µs	213 µs	388 µs	1.28 ms	15 ms	984 ms	479k	491k
Find by ID	25,800	39 µs	37 µs	77 µs	192 µs	717 µs	2.27 ms	85k	85k
Find earliest expiry	34,500	29 µs	27 µs	59 µs	147 µs	500 µs	22 ms	63k	63k
Delete by ID	4,100	241 µs	230 µs	403 µs	1.16 ms	16 ms	26 ms	518k	525k
Delete by expiry	4,600	216 µs	205 µs	366 µs	1.13 ms	15 ms	17 ms	462k	469k
State transition	2,500	404 µs	384 µs	672 µs	2.00 ms	22 ms	36 ms	868k	878k
Recycle storage	2,500	404 µs	388 µs	672 µs	1.76 ms	18 ms	31 ms	871k	879k

Isolated phase, rws run (readers READ_WRITE_SHARED)

Operation	txn/s	mean	p50	p99	p99.9	p99.99	max	thread cycles	process cycles
Insert	4,000	251 µs	211 µs	414 µs	4.78 ms	21 ms	1.02 s	498k	511k
Find by ID	24,200	41 µs	38 µs	90 µs	265 µs	1.09 ms	9.07 ms	90k	90k
Find earliest expiry	35,700	28 µs	26 µs	57 µs	188 µs	806 µs	2.95 ms	61k	61k
Delete by ID	4,000	247 µs	233 µs	422 µs	1.40 ms	16 ms	191 ms	527k	534k
Delete by expiry	4,600	219 µs	205 µs	377 µs	1.19 ms	16 ms	33 ms	469k	475k
State transition	2,400	408 µs	388 µs	724 µs	2.00 ms	21 ms	35 ms	879k	889k
Recycle storage	2,500	403 µs	384 µs	672 µs	1.85 ms	19 ms	25 ms	869k	877k

Isolated phase, rws-ckpt run (no checkpoints in this phase)

Operation	txn/s	mean	p50	p99	p99.9	p99.99	max	thread cycles	process cycles
Insert	4,500	220 µs	203 µs	369 µs	1.12 ms	14 ms	1.03 s	460k	472k
Find by ID	26,300	38 µs	36 µs	69 µs	162 µs	724 µs	3.07 ms	83k	83k
Find earliest expiry	35,800	28 µs	26 µs	55 µs	140 µs	605 µs	2.72 ms	61k	61k
Delete by ID	4,100	243 µs	230 µs	414 µs	1.28 ms	16 ms	47 ms	520k	527k
Delete by expiry	4,600	216 µs	203 µs	362 µs	1.19 ms	15 ms	19 ms	462k	469k
State transition	2,400	409 µs	384 µs	761 µs	2.72 ms	22 ms	65 ms	874k	884k
Recycle storage	2,500	402 µs	384 µs	664 µs	2.15 ms	19 ms	45 ms	864k	873k

Cycles are CPU cycles per transaction: thread = the calling thread; process = the whole process, including MemDB2's journal thread.

Mixed phase, read run (readers READ_SHARED)

Operation	txns	txn/s	mean	p50	p99	p99.9	p99.99	max	misses
Insert	47,795	1,500	1.07 ms	612 µs	3.49 ms	31 ms	82 ms	159 ms	0
Find by ID	682,162	21,400	169 µs	84 µs	1.60 ms	2.96 ms	20 ms	254 ms	291,627
Find earliest expiry	75,796	2,400	158 µs	74 µs	1.58 ms	3.28 ms	20 ms	160 ms	0
Delete by ID	23,889	700	1.44 ms	1.63 ms	3.88 ms	23 ms	157 ms	158 ms	10,118
Delete by expiry	28,589	900	769 µs	73 µs	3.28 ms	19 ms	90 ms	157 ms	28,589
State transition	11,884	400	1.32 ms	903 µs	4.00 ms	32 ms	99 ms	156 ms	5,169
Recycle storage	4,808	200	901 µs	86 µs	3.34 ms	8.60 ms	157 ms	157 ms	4,808

Mixed phase, rws run (readers READ_WRITE_SHARED)

Operation	txns	txn/s	mean	p50	p99	p99.9	p99.99	max	misses
Insert	89,506	2,900	800 µs	754 µs	1.49 ms	8.24 ms	54 ms	125 ms	0
Find by ID	628,735	20,100	179 µs	155 µs	508 µs	799 µs	5.79 ms	36 ms	268,633
Find earliest expiry	69,861	2,200	171 µs	147 µs	493 µs	799 µs	5.55 ms	30 ms	0
Delete by ID	44,774	1,400	580 µs	575 µs	1.42 ms	4.06 ms	33 ms	87 ms	18,831
Delete by expiry	53,944	1,700	190 µs	162 µs	582 µs	985 µs	5.91 ms	22 ms	53,944
State transition	22,263	700	688 µs	784 µs	1.58 ms	6.33 ms	30 ms	67 ms	9,596
Recycle storage	9,017	300	196 µs	166 µs	597 µs	985 µs	20 ms	20 ms	9,017

Mixed phase, rws-ckpt run (async checkpoints running)

Operation	txns	txn/s	mean	p50	p99	p99.9	p99.99	max	misses
Insert	40,694	1,300	1.88 ms	664 µs	27 ms	46 ms	141 ms	1.35 s	0
Find by ID	442,501	14,000	259 µs	101 µs	2.72 ms	27 ms	37 ms	64 ms	189,201
Find earliest expiry	49,166	1,600	241 µs	88 µs	1.78 ms	26 ms	43 ms	83 ms	0
Delete by ID	20,351	600	1.26 ms	411 µs	22 ms	41 ms	317 ms	599 ms	8,647
Delete by expiry	24,469	800	314 µs	101 µs	6.99 ms	27 ms	35 ms	38 ms	24,469
State transition	10,142	300	1.49 ms	649 µs	25 ms	45 ms	68 ms	102 ms	4,303
Recycle storage	4,068	100	331 µs	101 µs	7.34 ms	28 ms	45 ms	45 ms	4,068

Source: `Common\MemDB2\Bench\results.csv` and `results_execution.log`, 11 November 2026.